

Enhancement & Security Report

Comprehensive audit of **packages/server** and **packages/web** — 19 findings across security, code quality, UX, and performance.

Executive Summary. A full-codebase review dated 2026-06-30 identified **six critical security vulnerabilities** that require immediate remediation before any public deployment — notably an admin-secret bypass that grants unauthenticated callers full admin access, an OAuth token leak via redirect URL, and unguarded cross-room WebRTC relay. Beyond security, the server-side handler module has grown to 2,513 lines with no domain separation, creating compounding maintenance risk. Several UX defects — permanently hung button state and a double-firing phase-transition banner — degrade the player experience on network instability. All 19 findings are prioritised below; the top 6 are security-critical and should block release.

RELEASE BLOCKER — Six critical vulnerabilities are present in the current build. Do not expose this server to untrusted networks until C1-C6 are resolved. Any socket client can reach admin endpoints today.

SECURITY

Vulnerabilities

C1 Admin secret bypassed when env var is unset **CRITICAL**

`packages/server/src/config.ts:104–106`

If `ADMIN_SECRET` is not set, `isValidAdminSecret()` returns `true` for every caller. Any socket client can emit admin events (data export, impersonation, server stats) and receive full admin access.

Fix: Invert the default — deny all admin requests when `ADMIN_SECRET` is unconfigured and log a startup warning. Use `crypto.timingSafeEqual` to close the timing-attack vector.

C2 OAuth redirect embeds session token in URL query parameter **CRITICAL**

packages/server/src/createApp.ts:321

The full session token is appended as `?auth_token=<token>`. This leaks into browser history, server access logs, referrer headers, and proxy logs. The `httpOnly` cookie is set simultaneously, making the URL parameter redundant.

Fix: Remove the `auth_token` query parameter from the redirect. Update `packages/web/src/net/socket.ts:71–80` (`captureAuthTokenFromUrl`) to remove its consumer.

C3 Voice signaling relayed without room membership check **CRITICAL**

packages/server/src/handlers.ts:1063–1102

A logged-in player who knows another player's `playerId` can send arbitrary WebRTC offers, answers, and ICE candidates to that player's socket regardless of whether they share a room.

Fix: Before relaying, verify `session.roomCode === target.roomCode`. Reject silently otherwise. Add a targeted test case.

C4 Online-presence broadcast leaks to all connected clients globally **CRITICAL**

packages/server/src/handlers.ts:537, 549

`PLAYER_ONLINE_STATUS` is emitted with `socket.broadcast.emit`, delivering every user's online/offline state to every connected socket — including strangers in unrelated rooms.

Fix: Replace with `io.to(session.roomCode).emit(...)`. One-line change; zero functional impact on room members.

C5 Ghost-session adoption trusts client-supplied `playerId` **CRITICAL**

packages/server/src/handlers.ts:428–465

`tryAdoptGhostSession` accepts `incomingPlayerId` from the socket handshake auth with no proof of ownership. A guest client can supply any victim's `playerId` and take over their in-progress session during the reconnection grace period.

Fix: Require that the `guestToken` in socket auth matches the token stored in the ghost session, in addition to the `playerId` check.

C6 Room capacity not enforced on invite accept **CRITICAL**

packages/server/src/handlers.ts:1465–1466

`handleRespondToInvite` adds an accepting player without checking `room.players.length < room.maxPlayers`. Sending concurrent invites can overflow a 4-player room to 5+ players.

Fix: Add a capacity guard before the push: `if (room.players.length >= room.maxPlayers) { ack({ ok: false, error: 'Room is full' }); return; }`. Cover with a test.

M3 sanitizePlayerName allowlist too narrow **MAJOR**

packages/server/src/handlers.ts:130–140

Only `<>` are stripped. Characters such as `"`, `'`, backticks, backslashes, braces, and Unicode control characters pass through unchecked.

Fix: Use an allowlist regex `/^[a-zA-Z0-9 _\-'!?.!]+$/`. Strip non-printable/non-ASCII characters; enforce a 20-character limit; reject if nothing valid remains.

M6 `generateRoomCode` uses non-cryptographic `Math.random()` **MAJOR**

`packages/server/src/handlers.ts:2480–2488`

Room codes are 6-character identifiers that grant game access. `Math.random()` output is predictable; an adversary observing a few codes can predict future ones and join rooms uninvited.

Fix: Use `crypto.randomBytes(6)` and map each byte to a character from a safe unambiguous alphabet (e.g. `ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789`).

M7 **Missing HTTP security headers** **MAJOR**

`packages/server/src/createApp.ts`

No `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Strict-Transport-Security`, or `Referrer-Policy` headers are set.

Fix: Install `helmet` and add `app.use(helmet({ contentSecurityPolicy: { ... } })))`. One line eliminates clickjacking, MIME sniffing, and referrer leakage.

M8 **Capture array unbounded in move validator** **MAJOR**

`packages/server/src/handlers.ts:2508–2511`

`isMakeMovePayload` checks that `capture` is an array of strings but does not bound its length. A client can send thousands of entries, causing CPU amplification before the engine rejects the move.

Fix: Add `if ((m['capture'] as unknown[]).length > 52) return false;` — 52 is the maximum possible cards in a standard deck.

CODE QUALITY & REFACTORING

Structural Issues

M2 `handlers.ts` is a 2,513-line monolith **MAJOR**

`packages/server/src/handlers.ts` (entire file)

All 59 server-side event handlers — rooms, moves, invites, OAuth, admin, voice, scoring — live in one file. This makes focused unit testing, navigation, and safe modification extremely difficult.

Fix: Split by domain: `handlers/room.ts`, `handlers/game.ts`, `handlers/invite.ts`, `handlers/admin.ts`, `handlers/voice.ts`, `handlers/account.ts`. Each exports a `register(io, socket, session)` function; the root file becomes a thin orchestrator.

M4 `updateSession` silently ignores `userId` and `account` patches **MAJOR**

`packages/server/src/store.ts:131–138`

The function signature accepts `Partial<Omit<SessionState, 'token' | 'playerId'>>` but the implementation only applies `name`, `roomCode`, and `socketId`. Callers that pass `userId` or `account` believe they are persisting data; they are silently no-oped.

Fix: Either expand the implementation to apply all non-readonly fields, or narrow the accepted type to `{ name?: string; roomCode?: string | null; socketId?: string | null }`.

M10 **as never type cast in GameScreen** **MAJOR**

packages/web/src/screens/GameScreen.tsx:205

`handState.onSelect(id as never)` uses a type-erasure cast that bypasses TypeScript's type checker entirely, indicating a structural mismatch between `Part1SelectionState` and `Part2SelectionState`.

Fix: Define a shared `onSelect: (id: CardId) => void` on a common base type, or use a discriminated union, and eliminate the `as never` cast.

M11 **No test coverage for invite flow, voice signaling, or admin events** **MAJOR**

packages/server/src/handlers.test.ts

The test suite covers room creation, move validation, and game flow, but has zero coverage for `handleRespondToInvite` (capacity bypass), voice relay (missing room check), or admin event authentication.

Fix: Add test cases for: (a) invite accept when room is at max capacity, (b) voice relay to player in a different room, (c) admin events with and without `ADMIN_SECRET` configured.

m1 **Legacy token takes priority over fresh session token in bootstrapAuth** **MINOR**

packages/web/src/net/socket.ts:132

`getLegacyToken() ?? getSessionToken()` — an old `localStorage` token silently wins over a fresh `sessionStorage` token. On a shared device, subsequent visitors may be authenticated as the previous user.

Fix: Invert the fallback order so the session-scoped token takes priority, or explicitly clear the legacy token before reading.

m2 **`clearRuntimeSessionStorage` operates on `localStorage` despite its name** **MINOR**

packages/web/src/net/socket.ts:118–124

The function name implies `sessionStorage` but it clears `localStorage`. The mismatch creates confusion about what state is cleared on tab close vs. full logout.

Fix: Rename to `clearAllLocalAuth` to accurately reflect that both storage layers are cleared.

MISSING FEATURES & UX GAPS

Player-Facing Issues

M1 `emitAck` promises never time out **MAJOR**

`packages/web/src/net/socket.ts:179–187`

The promise only ever resolves — it never rejects and never times out. On server crash or disconnect, the promise hangs forever, permanently locking button state (`submitting = true`) and blocking all further user actions.

Fix: Wrap with `Promise.race` against a `setTimeout` reject at 8,000 ms, or use socket.io's built-in `socket.timeout(8000).emit(...)`.

M5 **Phase-transition** `useEffect` fires multiple times **MAJOR**

`packages/web/src/screens/GameScreen.tsx:118–126`

`prevPhase.current = view.phase` is skipped due to an early return when the transition fires. On the next re-render (still in `PART_2`) the condition is true again — the transition banner fires a second time and a new 2,500 ms timer is started.

Fix: Update `prevPhase.current = view.phase` unconditionally before the early return, so the guard is accurate on the next render.

M9 **Part1Board** ignores stale `onSelectionChange` callback **MAJOR**

`packages/web/src/components/Part1Board.tsx:197`

An `eslint-disable-next-line react-hooks/exhaustive-deps` comment suppresses the missing-dependency warning. If `onSelectionChange` identity changes (parent re-renders with a new reference), the child never picks up the new callback — classic stale closure bug.

Fix: Wrap `onSelectionChange` in `useCallback` in the parent with stable deps, or add it to the `useEffect` dependency array.

m3 **Capture options recomputed synchronously on every confirm** **MINOR**

`packages/web/src/components/Part1Board.tsx:115`

`captureOptionsFor(card, view.table)` is called inside `confirmCard()` without memoization. For large tables, the DFS in `maxDisjointUnions` can introduce perceptible latency on low-end devices.

Fix: Pre-compute capture options when a card is selected, using `useMemo(() => captureOptionsFor(card, table), [card, table])`.

m4 **bootstrapAuth** swallows all fetch errors silently **MINOR**

`packages/web/src/net/socket.ts:161`

The `catch` block is empty. If the bootstrap request fails, the app proceeds with stale auth state with no UI feedback for the user.

Fix: At minimum log the error; in production surface a "connection failed" banner so the user knows to retry.

`packages/web/src/components/Part1Board.tsx:55`

The `sessionStorage` key `ganatri_p1_sel_${'{'}}view.you{'}'}` is not scoped to the current room. When a player starts a new game in a new room, stale selection from the previous game can produce a spurious "card selected" UI state on mount.

Fix: Include the room code in the key: `ganatri_p1_sel_${'{'}}view.you{'}'}_${'{'}}view.roomCode ?? {'}'}`.

PRIORITISED ACTION PLAN

Top-10 Fixes by Impact

Items 1–6 are security-critical and block public release. Items 7–10 address code quality, stability, and performance in order of effort-to-impact ratio.

1 **Fix C1 — Admin auth default-deny**

Any deployment without `ADMIN_SECRET` set is fully exposed. Invert the default in `config.ts` and add `crypto.timingSafeEqual` comparison. One-file fix, zero test risk.

Highest-priority item — a single env var omission hands over full server control today.

2 **Fix C3 — Voice relay room membership check**

Add `session.roomCode === target.roomCode` guard in `handlers.ts:1063–1102` before relaying any WebRTC signal. One guard eliminates cross-room probing. Add a test.

3 **Fix C2 — Remove OAuth token from redirect URL**

Delete the `?auth_token=` parameter from `createApp.ts:321` and remove the matching consumer `captureAuthTokenFromUrl` in `socket.ts:71–80`. Eliminates token leakage into logs and browser history.

4 **Fix C4 — Scope presence events to room**

Replace `socket.broadcast.emit` with `io.to(session.roomCode).emit` at `handlers.ts:537, 549`. One-line change; no functional impact for legitimate room members.

5 **Fix C6 — Room capacity guard on invite accept**

Add `if (room.players.length >= room.maxPlayers)` check in `handleRespondToInvite`. Three lines prevent concurrent invite overflow. Cover with a test.

6

Fix C5 — Ghost-session adoption requires `guestToken`

In `tryAdoptGhostSession` require that the `guestToken` supplied in socket auth matches the token stored in the ghost session — not just the `playerId`. Prevents session hijacking during the reconnection grace period.

7

Fix M6 — Replace `Math.random()` with `randomBytes` for room codes

Swap `Math.random()` in `generateRoomCode` for `crypto.randomBytes(6)` mapped to an unambiguous alphabet. Ten-line change; no functional impact on room access flow.

8

Fix M1 — Add timeout to `emitAck`

Use `socket.timeout(8000).emit(...)` or `Promise.race` with a reject timer. Prevents permanently hung UI state on all user-facing action buttons after any network interruption.

9

Fix M7 — Add HTTP security headers via `helmet`

Install `helmet` and add `app.use(helmet({ contentSecurityPolicy: { ... } }))` in `createApp.ts`. Eliminates clickjacking, MIME sniffing, and referrer leakage in one call.

10

Fix M2 — Split `handlers.ts` into domain modules

Refactor the 2,513-line monolith into `handlers/room.ts`, `handlers/game.ts`, `handlers/invite.ts`, `handlers/admin.ts`, `handlers/voice.ts`, and `handlers/account.ts`. Larger effort but unlocks safe parallel development and targeted test coverage for the invite, admin, and voice paths.

This is a prerequisite for sustainably adding any new server-side feature going forward.

→ Severity labels: **Critical** = active exploit vector; **Major** = significant defect or maintenance hazard; **Minor** = polish or low-probability edge case. All findings were identified by static review on 2026-06-30.